

1. Theory of Constraints

Constraints are rules enforced on data columns in a table to maintain data integrity and accuracy. They prevent invalid data entry and ensure the database reflects the real world correctly.

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each row; cannot be NULL
FOREIGN KEY	Ensures referential integrity between tables
UNIQUE	Ensures all values in a column are different
CHECK	Validates data based on a logical expression
NOT NULL	Prevents NULL values in a column
DEFAULT	Provides a default value when no value is supplied

2. Minimal Code & Real-World Queries for Each Constraint

□ PRIMARY KEY

Concept: Every customer must have a unique ID.

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

Real-world query: Find customer by ID.

```
SELECT * FROM customers WHERE customer_id = 101;
```

● FOREIGN KEY

Concept: Orders must reference an existing customer.

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
FOREIGN KEY (columnName -> PK) REFERENCES parentTableName(columnName -> PK)  
);
```

Real-world query: Get all orders for customer #101.

```
SELECT o.order_id, c.name  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
WHERE c.customer_id = 101;
```

□ UNIQUE

Concept: No two users can have the same email.

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY,  
    email VARCHAR(100) UNIQUE  
);
```

Real-world query: Check if an email already exists.

```
SELECT COUNT(*) FROM users WHERE email = 'john@example.com';
```

● CHECK

Concept: Age must be ≥ 18 .

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    age INT CHECK (age >= 18)  
);
```

Real-world query: List employees eligible for retirement (age ≥ 65).

```
SELECT * FROM employees WHERE age >= 65;
```

● NOT NULL

Concept: Product name cannot be missing.

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL  
);
```

Real-world query: Find products without a name (should return none if constraint works).

```
SELECT * FROM products WHERE product_name IS NULL;
```

□ DEFAULT

Concept: New orders default to 'pending' status.

```
CREATE TABLE order_status (  
    order_id INT PRIMARY KEY,  
    country VARCHAR(20) DEFAULT 'Pakistan'  
);
```

Real-world query: Count orders not yet processed (still pending).

```
SELECT COUNT(*) FROM order_status WHERE status = 'pending';
```

3. Massive Data Example: Create Database, Table, Insert Bulk Data & Apply Operations

Step 1: Create Database

```
CREATE DATABASE ConstraintDemo;  
USE ConstraintDemo;
```

Step 2: Create Table with Multiple Constraints

```
CREATE TABLE regions (  
    region_name VARCHAR(50) PRIMARY KEY  
);  
  
INSERT INTO regions VALUES ('North'), ('South'), ('East'), ('West');  
  
CREATE TABLE sales (  
    sale_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_code VARCHAR(20) NOT NULL UNIQUE,  
    sale_date DATE NOT NULL,  
    quantity INT CHECK (quantity > 0),  
    price DECIMAL(10,2) CHECK (price >= 0),  
    region VARCHAR(50) DEFAULT 'Unknown',  
    FOREIGN KEY (region) REFERENCES regions(region_name) -- assuming regions table exists  
);  
  
-- For simplicity, we skip FOREIGN KEY in bulk insert.
```

Step 3: Insert Massive Data (1 million rows) using a stored procedure

```
DELIMITER $$  
CREATE PROCEDURE InsertMassiveSales()  
BEGIN  
    DECLARE i INT DEFAULT 1;  
    WHILE i <= 10000 DO  
        INSERT INTO sales (product_code, sale_date, quantity, price, region)  
        VALUES (  
            CONCAT('PROD-', LPAD(i, 7, '0')),  
            DATE_ADD('2020-01-01', INTERVAL FLOOR(RAND() * 2000) DAY),  
            FLOOR(1 + RAND() * 100),  
            ROUND(RAND() * 500, 2),  
            ELT(1 + FLOOR(RAND() * 4), 'North', 'South', 'East', 'West')  
        );  
        SET i = i + 1;  
    END WHILE;  
END
```

```
END$$  
DELIMITER ;  
  
CALL InsertMassiveSales();
```

Step 4: Apply Operations on the Massive Data

✓ Count total rows

```
SELECT COUNT(*) FROM sales; -- 1,000,000
```

✓ Query with constraint usage (e.g., CHECK)

Find sales where quantity is invalid (should return 0 if CHECK works).

```
SELECT * FROM sales WHERE quantity <= 0;
```

✓ Update with constraint validation

Increase price by 10% for all 'East' region, but ensure price stays ≥ 0 (CHECK).

```
UPDATE sales SET price = price * 1.10 WHERE region = 'East';
```

✓ Delete with constraint reference

Delete a product and see if any FK violations (if FK existed).

```
DELETE FROM sales WHERE product_code = 'PROD-0000001';
```

✓ Aggregation with group by (real-world analysis)

```
SELECT region, SUM(quantity * price) AS total_revenue  
FROM sales  
WHERE sale_date BETWEEN '2021-01-01' AND '2021-12-31'  
GROUP BY region  
ORDER BY total_revenue DESC;
```

✓ Constraint violation example (try to insert NULL in NOT NULL column)

```
INSERT INTO sales (product_code, sale_date, quantity, price) VALUES (NULL, '2023-01-01', 5, 10);
```

-- Error: Column 'product_code' cannot be null

Key Takeaways

- Constraints enforce data quality **at the database level**, reducing application logic.
- The massive data example shows how constraints scale with large datasets (performance impact minimal).
- Always design constraints based on real-world rules: e.g., `quantity > 0` prevents negative inventory.
- Use `CHECK` for simple business rules, `FOREIGN KEY` for relationships, and `UNIQUE` to avoid